

# Tests for variable importance estimation

Stacy Cyrille

2026-05-28

## Table of contents

|          |                                                                                   |           |
|----------|-----------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Scenario 1: Additive hazards</b>                                               | <b>2</b>  |
| 1.1      | Objectives . . . . .                                                              | 2         |
| 1.2      | Data generating mechanism . . . . .                                               | 2         |
| 1.3      | Variable importance . . . . .                                                     | 5         |
| 1.3.1    | Global survival stacking . . . . .                                                | 6         |
| 1.3.2    | Aalen model . . . . .                                                             | 8         |
| <b>2</b> | <b>Scenario 2: Mixed additive and multiplicative hazards</b>                      | <b>11</b> |
| 2.1      | Objectives . . . . .                                                              | 11        |
| 2.2      | Data-generating mechanism . . . . .                                               | 11        |
| 2.3      | Variable importance . . . . .                                                     | 14        |
| 2.3.1    | Global survival stacking . . . . .                                                | 15        |
| 2.3.2    | Cox-Aalen model . . . . .                                                         | 18        |
| <b>3</b> | <b>Scenario 3: Rare genomic alterations with mixed effects under proportional</b> | <b>20</b> |
| 3.1      | Objectives . . . . .                                                              | 20        |
| 3.2      | Data generating mechanism . . . . .                                               | 20        |
| 3.3      | Variable importance . . . . .                                                     | 23        |
| <b>4</b> | <b>Additional numerical experiments</b>                                           | <b>27</b> |
| 4.1      | Sensitivity to covariate correlations . . . . .                                   | 27        |
| 4.2      | Exclusion versus permutation variable importance . . . . .                        | 27        |

```
library(tidyverse)
library(MASS)
library(knitr)
library(survival)
library(timereg)
```

```

library(survML)
library(survivalSL)
library(SuperLearner)
library(randomForestSRC)
library(survex)

source(here::here("R", "utils.R"))
source(here::here("R", "generate_data.R"))
source(here::here("R", "pred_generators.R"))

# Set seed for reproducibility
set.seed(2026)
# n <- 200000
n <- 500
V <- 5
sample_split <- TRUE
vims <- c("AUC", "brier")

```

## 1 Scenario 1: Additive hazards

### 1.1 Objectives

This scenario:

- evaluates the overall performance of the VIM procedure under additive hazards: bias, confidence interval coverage, confidence interval width, rejection probability
- assesses the robustness of the VIM procedure to nuisance-model choice required for VIM computation: Aalen model (`timereg::aalen`), survival Super Learner (`survivalSL::survivalSL`), global survival stacking (`survML::stackG`)

### 1.2 Data generating mechanism

Four binary covariates were generated *independently* with prevalences spanning rare to frequent alterations,  $X_j \sim \text{Bernoulli}(\pi_j)$  with  $\pi_j = (0.005, 0.02, 0.05, 0.20)$ .

Events times were simulated under the following time-constant additive hazards model:

$$\lambda(t|x) = \lambda_0(t) + \beta_1(t)X_1 + \beta_2(t)X_2 + \beta_3(t)X_3 + \beta_4(t)X_4$$

with  $\lambda_0(t) = 0.05$  (constant) and  $\beta_j(t) = (0.06, 0.02, 0.005, 0)$ .

Independent censoring times were generated as  $C \sim \text{Uniform}(0, c_{\max})$ . The parameter  $c_{\max}$  was calibrated using a large Monte Carlo sample from the data-generating mechanism to achieve an expected censoring proportion of approximately 20%, and the calibrated value was then fixed across all simulation replicates.

```
p <- 4
sigma <- diag(x = 1, nrow = p, ncol = p)
# prev <- c(0.005, 0.02, 0.05, 0.20)
prev <- runif(p, 0.2, 0.4)
beta <- c(0.06, 0.02, 0.005, 0)
lambda0 <- 0.05
target_cens <- 0.2
c_max <- calibrate_c_max_aalen(lambda0      = lambda0,
                                beta         = beta,
                                prev         = prev,
                                sigma        = sigma,
                                target_cens  = target_cens,
                                seed         = 2026)
c_max
```

```
[1] 72.64886
```

```
data_scen1 <- simulate_aalen_data(n, prev, sigma, lambda0, beta, c_max)
head(data_scen1)
```

|   | time      | event | X1 | X2 | X3 | X4 |
|---|-----------|-------|----|----|----|----|
| 1 | 15.798980 | 1     | 0  | 0  | 0  | 1  |
| 2 | 16.570986 | 1     | 0  | 1  | 0  | 1  |
| 3 | 1.454993  | 1     | 0  | 0  | 0  | 0  |
| 4 | 21.854079 | 1     | 1  | 1  | 0  | 0  |
| 5 | 25.834806 | 1     | 0  | 0  | 0  | 0  |
| 6 | 26.417640 | 1     | 0  | 0  | 1  | 0  |

```
# Check prevalences
round(colMeans(data_scen1 %>% dplyr::select(starts_with("X"))), 3)
```

| X1    | X2    | X3    | X4    |
|-------|-------|-------|-------|
| 0.362 | 0.350 | 0.232 | 0.264 |

```
# Check censoring rate
mean(1 - data_scen1$event)
```

```
[1] 0.16
```

```
aalen_fit <- timereg::aalen(Surv(time, event)~X1+X2+X3+X4, data = data_scen1)
summary(aalen_fit)
```

Additive Aalen Model

Test for nonparametric terms

Test for non-significant effects

|             | Supremum-test of significance | p-value | H <sub>0</sub> : B(t)=0 |
|-------------|-------------------------------|---------|-------------------------|
| (Intercept) | 9.29                          | 0.000   |                         |
| X1          | 5.50                          | 0.000   |                         |
| X2          | 2.77                          | 0.112   |                         |
| X3          | 1.81                          | 0.753   |                         |
| X4          | 1.68                          | 0.790   |                         |

Test for time invariant effects

|             | Kolmogorov-Smirnov test | p-value | H <sub>0</sub> :constant effect |
|-------------|-------------------------|---------|---------------------------------|
| (Intercept) | 0.830                   | 0.000   |                                 |
| X1          | 0.983                   | 0.002   |                                 |
| X2          | 0.450                   | 0.151   |                                 |
| X3          | 0.489                   | 0.330   |                                 |
| X4          | 0.223                   | 0.899   |                                 |

|             | Cramer von Mises test | p-value | H <sub>0</sub> :constant effect |
|-------------|-----------------------|---------|---------------------------------|
| (Intercept) | 9.490                 | 0.000   |                                 |
| X1          | 10.100                | 0.002   |                                 |
| X2          | 1.740                 | 0.134   |                                 |
| X3          | 1.700                 | 0.396   |                                 |
| X4          | 0.342                 | 0.866   |                                 |

Call:

```
timereg::aalen(formula = Surv(time, event) ~ X1 + X2 + X3 + X4,
  data = data_scen1)
```

```
time <- data_scen1$time
event <- data_scen1$event
X <- data_scen1 %>% dplyr::select(starts_with("X"))
```

### 1.3 Variable importance

We compare the following approaches for estimating the nuisance functions required for VIM computation:

- Aalen additive hazards model (`timereg::aalen`)
- Super Learner for survival prediction (`survivalSL::survivalSL`)
- Global survival stacking (`survML::stackG`)

We define time horizons, maximum follow-up time and times at which to approximate integrals.

```
landmark_times <- quantile(time, c(0.25, 0.75))
approx_times <- sort(unique(c(0, time[event == 1], landmark_times)))
approx_times <- approx_times[approx_times <= max(landmark_times)]
length(approx_times)
```

```
[1] 322
```

We generated cross-fitting and sample-splitting folds.

```
folds <- survML::generate_folds(n, V, sample_split)
cf_folds <- folds$cf_folds
table(cf_folds)
```

```
cf_folds
 1  2  3  4  5  6  7  8  9 10
50 50 50 50 50 50 50 50 50 50
```

```
ss_folds <- folds$ss_folds
table(ss_folds)
```

```
ss_folds
 0  1
250 250
```

### 1.3.1 Global survival stacking

```
VO_stackG_preds_scen1 <- CV_generate_full_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  approx_times = approx_times,
  nuisance = "stackG",
  cf_folds = cf_folds)
# str(VO_stackG_preds_scen1)

CV_full_preds <- VO_stackG_preds_scen1$CV_full_preds
CV_full_preds_train <- VO_stackG_preds_scen1$CV_full_preds_train
CV_S_preds <- VO_stackG_preds_scen1$CV_S_preds
CV_G_preds <- VO_stackG_preds_scen1$CV_G_preds

indx_s <- 1:ncol(X)
vim_stackG_scen1 <- purrr::map_dfr(indx_s, function(indx_i) {

  char_indx <- as.character(indx_i)
  indx <- as.numeric(strsplit(char_indx, ",")[[1]])

  CV_reduced_preds <- CV_generate_reduced_predictions_landmark(
    time = time,
    event = event,
    X = X,
    landmark_times = landmark_times,
    cf_folds = cf_folds,
    indx = indx,
    full_preds_train = CV_full_preds_train
  )

  purrr::map_dfr(vims, function(vim) {

    output <- switch(
      vim,
      brier = survML::vim_brier(
        time = time,
        event = event,
        approx_times = approx_times,
        landmark_times = landmark_times,
```

```

    f_hat = CV_full_preds,
    fs_hat = CV_reduced_preds,
    S_hat = CV_S_preds,
    G_hat = CV_G_preds,
    cf_folds = cf_folds,
    sample_split = sample_split,
    ss_folds = ss_folds
  ),
  AUC = survML::vim_AUC(
    time = time,
    event = event,
    approx_times = approx_times,
    landmark_times = landmark_times,
    f_hat = purrr::map(CV_full_preds, ~ 1 - .x),
    fs_hat = purrr::map(CV_reduced_preds, ~ 1 - .x),
    S_hat = CV_S_preds,
    G_hat = CV_G_preds,
    cf_folds = cf_folds,
    sample_split = sample_split,
    ss_folds = ss_folds
  )
)

output %>%
  dplyr::mutate(
    vim = vim,
    indx = char_indx
  )
})
})

kable(vim_stackG_scen1 %>%
  dplyr::filter(vim == "AUC")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),
  col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large AUC", "small

```

| Variable | Horizon | VIM    | var(VIM) | CIL    | CIU   | p     | large AUC | small AUC |
|----------|---------|--------|----------|--------|-------|-------|-----------|-----------|
| 1        | 2.801   | 0.079  | 217.348  | -1.748 | 1.907 | 0.466 | 0.535     | 0.456     |
| 1        | 16.914  | 0.198  | 40.802   | -0.594 | 0.989 | 0.312 | 0.727     | 0.529     |
| 2        | 2.801   | -0.011 | 160.400  | -1.581 | 1.559 | 0.506 | 0.535     | 0.546     |
| 2        | 16.914  | 0.103  | 36.429   | -0.645 | 0.851 | 0.394 | 0.727     | 0.624     |
| 3        | 2.801   | -0.001 | 159.723  | -1.568 | 1.565 | 0.501 | 0.535     | 0.536     |
| 3        | 16.914  | 0.046  | 36.282   | -0.701 | 0.793 | 0.452 | 0.727     | 0.681     |
| 4        | 2.801   | 0.015  | 167.267  | -1.588 | 1.618 | 0.493 | 0.535     | 0.520     |
| 4        | 16.914  | 0.054  | 35.601   | -0.685 | 0.794 | 0.443 | 0.727     | 0.673     |

```
kable(vim_stackG_scen1 %>%
  dplyr::filter(vim == "brier")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),
  col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large Brier", "small Brier"))
```

| Variable | Horizon | VIM    | var(VIM) | CIL    | CIU   | p     | large Brier | small Brier |
|----------|---------|--------|----------|--------|-------|-------|-------------|-------------|
| 1        | 2.801   | 0.062  | 0.391    | -0.015 | 0.140 | 0.058 | -0.732      | -0.794      |
| 1        | 16.914  | -0.002 | 1.257    | -0.141 | 0.137 | 0.509 | -0.546      | -0.545      |
| 2        | 2.801   | 0.062  | 0.391    | -0.015 | 0.140 | 0.057 | -0.732      | -0.794      |
| 2        | 16.914  | 0.004  | 1.251    | -0.134 | 0.143 | 0.475 | -0.546      | -0.551      |
| 3        | 2.801   | 0.062  | 0.391    | -0.015 | 0.140 | 0.057 | -0.732      | -0.794      |
| 3        | 16.914  | 0.008  | 1.253    | -0.131 | 0.146 | 0.457 | -0.546      | -0.554      |
| 4        | 2.801   | 0.063  | 0.391    | -0.015 | 0.140 | 0.057 | -0.732      | -0.795      |
| 4        | 16.914  | 0.006  | 1.252    | -0.133 | 0.144 | 0.467 | -0.546      | -0.552      |

### 1.3.2 Aalen model

```
V0_aalen_preds_scen1 <- CV_generate_full_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  approx_times = approx_times,
  nuisance = "aalen",
```



```

    cf_folds = cf_folds)
# str(V0_aalen_preds_scen1)

CV_full_preds <- V0_aalen_preds_scen1$CV_full_preds
CV_full_preds_train <- V0_aalen_preds_scen1$CV_full_preds_train
CV_S_preds <- V0_aalen_preds_scen1$CV_S_preds
CV_G_preds <- V0_aalen_preds_scen1$CV_G_preds

indx_s <- 1:ncol(X)
vim_aalen_scen1 <- purrr::map_dfr(indxs, function(indx_i) {

  char_indx <- as.character(indx_i)
  indx <- as.numeric(strsplit(char_indx, split = ",")[[1]])

  CV_reduced_preds <- CV_generate_reduced_predictions_landmark(
    time = time,
    event = event,
    X = X,
    landmark_times = landmark_times,
    cf_folds = cf_folds,
    indx = indx,
    full_preds_train = CV_full_preds_train
  )

  purrr::map_dfr(vims, function(vim) {

    output <- switch(
      vim,
      brier = survML::vim_brier(
        time = time,
        event = event,
        approx_times = approx_times,
        landmark_times = landmark_times,
        f_hat = CV_full_preds,
        fs_hat = CV_reduced_preds,
        S_hat = CV_S_preds,
        G_hat = CV_G_preds,
        cf_folds = cf_folds,
        sample_split = sample_split,
        ss_folds = ss_folds
      ),
      AUC = survML::vim_AUC(

```

```

    time = time,
    event = event,
    approx_times = approx_times,
    landmark_times = landmark_times,
    f_hat = purrr::map(CV_full_preds, ~ 1 - .x),
    fs_hat = purrr::map(CV_reduced_preds, ~ 1 - .x),
    S_hat = CV_S_preds,
    G_hat = CV_G_preds,
    cf_folds = cf_folds,
    sample_split = sample_split,
    ss_folds = ss_folds
  )
)

output %>%
  dplyr::mutate(
    vim = vim,
    indx = char_indx
  )
})
})

kable(vim_aalen_scen1 %>%
  dplyr::filter(vim == "AUC")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),
  col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large AUC", "small

```

| Variable | Horizon | VIM   | var(VIM) | CIL    | CIU   | p     | large AUC | small AUC |
|----------|---------|-------|----------|--------|-------|-------|-----------|-----------|
| 1        | 2.801   | 0.127 | 0.910    | 0.009  | 0.246 | 0.017 | 0.598     | 0.471     |
| 1        | 16.914  | 0.136 | 0.746    | 0.029  | 0.244 | 0.006 | 0.666     | 0.530     |
| 2        | 2.801   | 0.068 | 0.917    | -0.051 | 0.186 | 0.132 | 0.598     | 0.530     |
| 2        | 16.914  | 0.024 | 0.734    | -0.082 | 0.130 | 0.329 | 0.666     | 0.642     |
| 3        | 2.801   | 0.050 | 0.915    | -0.069 | 0.169 | 0.204 | 0.598     | 0.548     |
| 3        | 16.914  | 0.020 | 0.695    | -0.083 | 0.123 | 0.352 | 0.666     | 0.646     |
| 4        | 2.801   | 0.070 | 0.969    | -0.052 | 0.192 | 0.131 | 0.598     | 0.528     |
| 4        | 16.914  | 0.013 | 0.712    | -0.092 | 0.118 | 0.404 | 0.666     | 0.653     |

```
kable(vim_aalen_scen1 %>%
  dplyr::filter(vim == "brier")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),
  col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large Brier", "small Brier"))
```

| Variable | Horizon | VIM    | var(VIM) | CIL    | CIU    | p     | large Brier | small Brier |
|----------|---------|--------|----------|--------|--------|-------|-------------|-------------|
| 1        | 2.801   | 0.021  | 0.121    | -0.022 | 0.065  | 0.164 | -0.487      | -0.508      |
| 1        | 16.914  | -0.057 | 0.099    | -0.096 | -0.018 | 0.998 | -0.410      | -0.353      |
| 2        | 2.801   | 0.026  | 0.125    | -0.018 | 0.070  | 0.123 | -0.487      | -0.513      |
| 2        | 16.914  | -0.015 | 0.124    | -0.059 | 0.028  | 0.752 | -0.410      | -0.395      |
| 3        | 2.801   | 0.026  | 0.126    | -0.018 | 0.070  | 0.123 | -0.487      | -0.513      |
| 3        | 16.914  | -0.012 | 0.126    | -0.056 | 0.032  | 0.710 | -0.410      | -0.398      |
| 4        | 2.801   | 0.027  | 0.125    | -0.017 | 0.071  | 0.115 | -0.487      | -0.514      |
| 4        | 16.914  | -0.013 | 0.126    | -0.057 | 0.031  | 0.714 | -0.410      | -0.398      |

## 2 Scenario 2: Mixed additive and multiplicative hazards

### 2.1 Objectives

This scenario:

- evaluates the overall performance of the VIM procedure under mixed additive and multiplicative hazards: bias, confidence interval coverage, confidence interval width, rejection probability
- assesses the robustness of the VIM procedure to nuisance-model choice required for VIM computation: Cox-Aalen model (`timereg::cox.aalen`), survival Super Learner (`survivalSL::survivalSL`), global survival stacking (`survML::stackG`)

### 2.2 Data-generating mechanism

The covariates are generated *independently* as follows:

- $X = (X_1, X_2, X_3, X_4)$
- $X_1, X_2 \sim \mathcal{B}(0.4)$
- $X_3, X_4 \sim \mathcal{N}(0, 1)$

To generate mixed additive and multiplicative effects, event times were simulated under a Cox-Aalen model with constant baseline hazard:  $\lambda_0(t) = 0.05$ . Additive effects were assigned to  $X_1$  and  $X_3$  with coefficients  $\beta_1(t) = 0.06$  and  $\beta_3(t) = 0.005$ , respectively. Multiplicative (proportional hazards) effects were assigned to  $X_2$  and  $X_4$  with coefficients  $\beta_2 = 0.5$  and  $\beta_4 = 0.2$ , corresponding to hazard ratios of approximately 1.65 and 1.22.

Independent censoring times were generated as  $C \sim \text{Uniform}(0, c_{\max})$ . The parameter  $c_{\max}$  was calibrated using a large Monte Carlo sample from the data-generating mechanism to achieve an expected censoring proportion of approximately 20%, and the calibrated value was then fixed across all simulation replicates.

```
prev_add <- c(0.4, 0.4)
sigma_add <- diag(x = 1, nrow = 2, ncol = 2)
beta_add <- c(0.06, 0.005)
prev_mult <- 0.4
sigma_mult_bin <- diag(x = 1, nrow = 1, ncol = 1)
mean_mult <- 0
sigma_mult_cont <- diag(x = 1, nrow = 1, ncol = 1)
beta_mult <- c(0.5, 0.2)
lambda0 <- 0.05
target_cens <- 0.2
c_max <- calibrate_c_max_cox_aalen(lambda0          = lambda0,
                                   beta_add          = beta_add,
                                   beta_mult         = beta_mult,
                                   prev_add          = prev_add,
                                   sigma_add         = sigma_add,
                                   prev_mult         = prev_mult,
                                   mean_mult         = mean_mult,
                                   sigma_mult_bin    = sigma_mult_bin,
                                   sigma_mult_cont   = sigma_mult_cont,
                                   target_cens       = target_cens,
                                   seed              = 2026)

c_max
```

```
[1] 63.50589
```

```
data_scen2 <- simulate_cox_aalen_data(
  n, lambda0, beta_add, beta_mult,
  prev_add, sigma_add, prev_mult, mean_mult,
  sigma_mult_bin, sigma_mult_cont, c_max
)
head(data_scen2)
```

|   | time      | event | X1 | X2 | X3 | X4          |
|---|-----------|-------|----|----|----|-------------|
| 1 | 14.799082 | 1     | 0  | 1  | 0  | -0.14964990 |
| 2 | 3.499728  | 1     | 1  | 1  | 1  | 2.79265615  |
| 3 | 1.312034  | 1     | 0  | 1  | 0  | 0.04056139  |
| 4 | 21.180384 | 1     | 1  | 0  | 1  | -1.50816900 |
| 5 | 10.240640 | 1     | 1  | 1  | 0  | 0.46224676  |
| 6 | 27.884512 | 0     | 0  | 0  | 0  | -0.23424115 |

```
# Check prevalences
colMeans(data_scen2 %>% dplyr::select(X1:X3))
```

|  | X1    | X2    | X3    |
|--|-------|-------|-------|
|  | 0.418 | 0.402 | 0.426 |

```
# Check mean and sd
round(mean(data_scen2$X4), 3)
```

```
[1] 0
```

```
sd(data_scen2$X4)
```

```
[1] 1
```

```
# Check censoring rate
mean(1 - data_scen2$event)
```

```
[1] 0.168
```

```
cox_aalen_fit <- timereg::cox.aalen(Surv(time, event)~X1+X2+prop(X3)+prop(X4),
                                   data = data_scen2, n.sim = 1000, basesim = 1)
summary(cox_aalen_fit)
```

Cox-Aalen Model

Test for Aalen terms

Test for nonparametric terms

Test for non-significant effects

Supremum-test of significance p-value  $H_0: B(t)=0$

|             |      |       |
|-------------|------|-------|
| (Intercept) | 8.92 | 0.000 |
| X1          | 5.21 | 0.000 |
| X2          | 2.78 | 0.117 |

Test for time invariant effects

|             | Kolmogorov-Smirnov test | p-value H <sub>0</sub> :constant effect |
|-------------|-------------------------|-----------------------------------------|
| (Intercept) | 0.666                   | 0.000                                   |
| X1          | 0.870                   | 0.000                                   |
| X2          | 0.611                   | 0.122                                   |

Proportional Cox terms :

|          | Coef. | SE Robust | SE D2log(L) <sup>-1</sup> | z      | P-val | lower2.5% | upper97.5% |
|----------|-------|-----------|---------------------------|--------|-------|-----------|------------|
| prop(X3) | 0.350 | 0.1030    | 0.0954                    | 0.1010 | 3.67  | 2.41e-04  | 0.148      |
| prop(X4) | 0.235 | 0.0566    | 0.0498                    | 0.0525 | 4.71  | 2.46e-06  | 0.124      |

Test of Proportionality

|          | sup  hat U(t) | p-value H <sub>0</sub> |
|----------|---------------|------------------------|
| prop(X3) | 9.49          | 0.224                  |
| prop(X4) | 14.30         | 0.559                  |

```
time <- data_scen2$time
event <- data_scen2$event
X <- data_scen2 %>% dplyr::select(starts_with("X"))
```

## 2.3 Variable importance

We compare the following approaches for estimating the nuisance functions required for VIM computation:

- Cox-Aalen model (`timereg::cox.aalen`)
- Survival Super Learner (`survivalSL::survivalSL`)
- Global survival stacking (`survML::stackG`)

We define time horizons, maximum follow-up time and times at which to approximate integrals.

```
landmark_times <- quantile(time, c(0.25, 0.75))
approx_times <- sort(unique(c(0, time[event == 1], landmark_times)))
approx_times <- approx_times[approx_times <= max(landmark_times)]
length(approx_times)
```

[1] 323

We generated cross-fitting and sample-splitting folds.

```
foldes <- survML::generate_folds(n, V, sample_split)
cf_folds <- foldes$cf_folds
table(cf_folds)
```

```
cf_folds
 1  2  3  4  5  6  7  8  9 10
50 50 50 50 50 50 50 50 50 50
```

```
ss_folds <- foldes$ss_folds
table(ss_folds)
```

```
ss_folds
 0   1
250 250
```

### 2.3.1 Global survival stacking

```
V0_stackG_preds_scen2 <- CV_generate_full_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  approx_times = approx_times,
  nuisance = "stackG",
  cf_folds = cf_folds)
# str(V0_stackG_preds_scen2)

CV_full_preds <- V0_stackG_preds_scen2$CV_full_preds
CV_full_preds_train <- V0_stackG_preds_scen2$CV_full_preds_train
CV_S_preds <- V0_stackG_preds_scen2$CV_S_preds
CV_G_preds <- V0_stackG_preds_scen2$CV_G_preds

indxes <- 1:ncol(X)
vim_stackG_scen2 <- purrr::map_dfr(indxes, function(indx_i) {

  char_indx <- as.character(indx_i)
  indx <- as.numeric(strsplit(char_indx, split = ",")[[1]])
```

```

CV_reduced_preds <- CV_generate_reduced_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  cf_folds = cf_folds,
  indx = indx,
  full_preds_train = CV_full_preds_train
)

purrr::map_dfr(vims, function(vim) {

  output <- switch(
    vim,
    brier = survML::vim_brier(
      time = time,
      event = event,
      approx_times = approx_times,
      landmark_times = landmark_times,
      f_hat = CV_full_preds,
      fs_hat = CV_reduced_preds,
      S_hat = CV_S_preds,
      G_hat = CV_G_preds,
      cf_folds = cf_folds,
      sample_split = sample_split,
      ss_folds = ss_folds
    ),
    AUC = survML::vim_AUC(
      time = time,
      event = event,
      approx_times = approx_times,
      landmark_times = landmark_times,
      f_hat = purrr::map(CV_full_preds, ~ 1 - .x),
      fs_hat = purrr::map(CV_reduced_preds, ~ 1 - .x),
      S_hat = CV_S_preds,
      G_hat = CV_G_preds,
      cf_folds = cf_folds,
      sample_split = sample_split,
      ss_folds = ss_folds
    )
  )
})

```



```

output %>%
  dplyr::mutate(
    vim = vim,
    indx = char_indx
  )
})
})

kable(vim_stackG_scen2 %>%
  dplyr::filter(vim == "AUC")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),
  col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large AUC", "small

```

| Variable | Horizon | VIM    | var(VIM) | CIL    | CIU   | p     | large AUC | small AUC |
|----------|---------|--------|----------|--------|-------|-------|-----------|-----------|
| 1        | 2.426   | 0.086  | 122.399  | -1.286 | 1.457 | 0.451 | 0.673     | 0.587     |
| 1        | 12.492  | 0.061  | 11.081   | -0.352 | 0.473 | 0.387 | 0.681     | 0.620     |
| 2        | 2.426   | 0.058  | 117.917  | -1.288 | 1.404 | 0.466 | 0.673     | 0.615     |
| 2        | 12.492  | -0.017 | 10.462   | -0.418 | 0.384 | 0.532 | 0.681     | 0.697     |
| 3        | 2.426   | 0.061  | 125.968  | -1.330 | 1.452 | 0.466 | 0.673     | 0.612     |
| 3        | 12.492  | -0.034 | 10.816   | -0.442 | 0.374 | 0.565 | 0.681     | 0.715     |
| 4        | 2.426   | 0.083  | 117.279  | -1.260 | 1.425 | 0.452 | 0.673     | 0.590     |
| 4        | 12.492  | 0.061  | 12.856   | -0.384 | 0.505 | 0.395 | 0.681     | 0.620     |

```

kable(vim_stackG_scen2 %>%
  dplyr::filter(vim == "brier")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),
  col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large Brier", "small

```

| Variable | Horizon | VIM    | var(VIM) | CIL    | CIU   | p     | large Brier | small Brier |
|----------|---------|--------|----------|--------|-------|-------|-------------|-------------|
| 1        | 2.426   | 0.026  | 0.393    | -0.051 | 0.104 | 0.253 | -0.747      | -0.774      |
| 1        | 12.492  | -0.071 | 0.579    | -0.166 | 0.023 | 0.931 | -0.575      | -0.503      |
| 2        | 2.426   | 0.027  | 0.393    | -0.051 | 0.105 | 0.247 | -0.747      | -0.774      |

| Variable | Horizon | VIM    | var(VIM) | CIL    | CIU   | p     | large Brier | small Brier |
|----------|---------|--------|----------|--------|-------|-------|-------------|-------------|
| 2        | 12.492  | -0.065 | 0.575    | -0.159 | 0.029 | 0.911 | -0.575      | -0.510      |
| 3        | 2.426   | 0.027  | 0.393    | -0.051 | 0.105 | 0.248 | -0.747      | -0.774      |
| 3        | 12.492  | -0.066 | 0.561    | -0.159 | 0.027 | 0.919 | -0.575      | -0.508      |
| 4        | 2.426   | 0.027  | 0.393    | -0.051 | 0.104 | 0.250 | -0.747      | -0.774      |
| 4        | 12.492  | -0.069 | 0.573    | -0.162 | 0.025 | 0.924 | -0.575      | -0.506      |

### 2.3.2 Cox-Aalen model

```

V0_cox_aalen_preds_scen2 <- CV_generate_full_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  approx_times = approx_times,
  nuisance = "cox.aalen",
  cf_folds = cf_folds)
# str(V0_cox_aalen_preds_scen2)

CV_full_preds <- V0_cox_aalen_preds_scen2$CV_full_preds
CV_full_preds_train <- V0_cox_aalen_preds_scen2$CV_full_preds_train
CV_S_preds <- V0_cox_aalen_preds_scen2$CV_S_preds
CV_G_preds <- V0_cox_aalen_preds_scen2$CV_G_preds

indx <- 1:ncol(X)
vim_cox_aalen_scen2 <- purrr::map_dfr(indxs, function(indx_i) {

  char_indx <- as.character(indx_i)
  indx <- as.numeric(strsplit(char_indx, split = ",")[[1]])

  CV_reduced_preds <- CV_generate_reduced_predictions_landmark(
    time = time,
    event = event,
    X = X,
    landmark_times = landmark_times,
    cf_folds = cf_folds,
    indx = indx,
    full_preds_train = CV_full_preds_train
  )
})

```

```

purrr::map_dfr(vims, function(vim) {

  output <- switch(
    vim,
    brier = survML::vim_brier(
      time = time,
      event = event,
      approx_times = approx_times,
      landmark_times = landmark_times,
      f_hat = CV_full_preds,
      fs_hat = CV_reduced_preds,
      S_hat = CV_S_preds,
      G_hat = CV_G_preds,
      cf_folds = cf_folds,
      sample_split = sample_split,
      ss_folds = ss_folds
    ),
    AUC = survML::vim_AUC(
      time = time,
      event = event,
      approx_times = approx_times,
      landmark_times = landmark_times,
      f_hat = purrr::map(CV_full_preds, ~ 1 - .x),
      fs_hat = purrr::map(CV_reduced_preds, ~ 1 - .x),
      S_hat = CV_S_preds,
      G_hat = CV_G_preds,
      cf_folds = cf_folds,
      sample_split = sample_split,
      ss_folds = ss_folds
    )
  )

  output %>%
    dplyr::mutate(
      vim = vim,
      indx = char_indx
    )
})

})

kable(vim_cox_aalen_scen2 %>%
  dplyr::filter(vim == "AUC")%>%

```

```

tibble::remove_rownames()%>%
mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
dplyr::select(-c(cil_1sided, vim))%>%
dplyr::relocate(indx, landmark_time),
col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large AUC", "small AUC")

kable(vim_cox_aalen_scen2 %>%
dplyr::filter(vim == "brier")%>%
tibble::remove_rownames()%>%
mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
dplyr::select(-c(cil_1sided, vim))%>%
dplyr::relocate(indx, landmark_time),
col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large Brier", "small Brier"))

```

### 3 Scenario 3: Rare genomic alterations with mixed effects under proportional

hazards

#### 3.1 Objectives

This scenario aims to evaluate the overall performance of the VIM procedure to *rare alterations* exhibiting *heterogeneous effect sizes (ranging from weak to strong)* under a *proportional hazards model*. More specifically, the objective is to assess whether the VIM approach can:

- detect rare yet highly prognostic alterations,
- discriminate between weak and strong prognostic signals, and
- correctly identify non-informative (null) variables.

#### 3.2 Data generating mechanism

All covariates were generated independently as binary variables:

$$X_j \sim \mathcal{B}(\pi_j)$$

with  $\pi \in \{0.01, 0.1\}$  to reflect genomic rarity.

The event times follows a proportional hazards model with a with constant baseline hazard  $\lambda_0(t) = 0.1$  such that

$$\lambda(t | X) = \lambda_0(t) \exp(X^\top \beta).$$

Covariates were partitioned according to both their prevalence and prognostic effect size, resulting in the following combinations:

| Alteration frequency       | Effect category | HR  | $\beta$     |
|----------------------------|-----------------|-----|-------------|
| Very rare ( $\pi = 0.01$ ) | Strong          | 3.0 | $\log(3)$   |
| Very rare ( $\pi = 0.01$ ) | Strong          | 2.0 | $\log(2)$   |
| Very rare ( $\pi = 0.01$ ) | Moderate        | 1.5 | $\log(1.5)$ |
| Very rare ( $\pi = 0.01$ ) | Weak            | 1.2 | $\log(1.2)$ |
| Very rare ( $\pi = 0.01$ ) | Null            | 1.0 | 0           |
| Rare ( $\pi = 0.10$ )      | Strong          | 3.0 | $\log(3)$   |
| Rare ( $\pi = 0.10$ )      | Strong          | 2.0 | $\log(2)$   |
| Rare ( $\pi = 0.10$ )      | Moderate        | 1.5 | $\log(1.5)$ |
| Rare ( $\pi = 0.10$ )      | Weak            | 1.2 | $\log(1.2)$ |
| Rare ( $\pi = 0.10$ )      | Null            | 1.0 | 0           |

This design allows us to assess whether the VIM procedure can detect rare but strongly prognostic alterations, distinguish weak from moderate and strong effects, and correctly assign low importance to null variables.

Independent censoring times were generated as  $C \sim \text{Uniform}(0, c_{\max})$ . The parameter  $c_{\max}$  was calibrated using a large Monte Carlo sample from the data-generating mechanism to achieve an expected censoring proportion of approximately 20%, and the calibrated value was then fixed across all simulation replicates.

```
p <- 10
prev <- c(rep(0.01, 5), rep(0.1, 5))
beta <- rep(c(log(3), log(2), log(1.5), log(1.2), 0), 2)
sigma <- diag(x = 1, nrow = p, ncol = p)
lambda0 <- 0.1
target_cens <- 0.2
c_max_scen3 <- calibrate_c_max_cox(lambda0 = lambda0,
                                   beta = beta,
                                   prev = prev,
                                   sigma = sigma,
                                   target_cens = target_cens,
                                   seed = 2026)

c_max_scen3
```

```
[1] 40.89694
```

```
data_scen3 <- simulate_cox_data(n, prev, sigma, lambda0, beta, c_max_scen3)
head(data_scen3)
```

```
      time event X1 X2 X3 X4 X5 X6 X7 X8 X9 X10
1 10.2834804     1  0  0  0  0  0  0  1  0  0  0
2  0.3443951     1  0  0  0  0  0  1  0  0  0  0
3  3.3076462     1  0  0  0  0  0  0  0  1  0  0
4  3.4743000     0  0  0  0  0  0  0  0  0  0  0
5 15.6715162     1  0  0  0  0  0  0  0  0  1  0
6  8.8225439     1  0  0  0  0  0  0  0  0  0  1
```

```
# Check empirical prevalences
round(colMeans(data_scen3 %>% dplyr::select(starts_with("X"))), 3)
```

```
      X1      X2      X3      X4      X5      X6      X7      X8      X9      X10
0.006 0.012 0.008 0.006 0.010 0.102 0.098 0.118 0.096 0.104
```

```
# Check censoring
mean(1 - data_scen3$event)
```

```
[1] 0.192
```

```
# Cox PH model
coxph_fit <- coxph(Surv(time, event)~., data_scen3)
summary(coxph_fit)
```

Call:

```
coxph(formula = Surv(time, event) ~ ., data = data_scen3)
```

```
n= 500, number of events= 404
```

|    | coef   | exp(coef) | se(coef) | z     | Pr(> z )     |
|----|--------|-----------|----------|-------|--------------|
| X1 | 0.6478 | 1.9113    | 0.5820   | 1.113 | 0.265701     |
| X2 | 0.7097 | 2.0334    | 0.4624   | 1.535 | 0.124821     |
| X3 | 1.3408 | 3.8219    | 0.5910   | 2.269 | 0.023288 *   |
| X4 | 0.8246 | 2.2809    | 0.5862   | 1.407 | 0.159541     |
| X5 | 1.1361 | 3.1146    | 0.4534   | 2.506 | 0.012213 *   |
| X6 | 1.4672 | 4.3370    | 0.1662   | 8.830 | < 2e-16 ***  |
| X7 | 0.5836 | 1.7925    | 0.1682   | 3.470 | 0.000521 *** |

```

X8    0.2681    1.3075    0.1577    1.700 0.089156 .
X9    0.1631    1.1772    0.1627    1.002 0.316110
X10 -0.2531    0.7764    0.1712   -1.478 0.139330
---
```

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

|     | exp(coef) | exp(-coef) | lower .95 | upper .95 |
|-----|-----------|------------|-----------|-----------|
| X1  | 1.9113    | 0.5232     | 0.6108    | 5.980     |
| X2  | 2.0334    | 0.4918     | 0.8215    | 5.033     |
| X3  | 3.8219    | 0.2616     | 1.2001    | 12.171    |
| X4  | 2.2809    | 0.4384     | 0.7230    | 7.196     |
| X5  | 3.1146    | 0.3211     | 1.2809    | 7.574     |
| X6  | 4.3370    | 0.2306     | 3.1315    | 6.007     |
| X7  | 1.7925    | 0.5579     | 1.2891    | 2.493     |
| X8  | 1.3075    | 0.7648     | 0.9598    | 1.781     |
| X9  | 1.1772    | 0.8495     | 0.8557    | 1.619     |
| X10 | 0.7764    | 1.2880     | 0.5551    | 1.086     |

Concordance= 0.609 (se = 0.014 )

Likelihood ratio test= 79.78 on 10 df, p=6e-13

Wald test = 98.78 on 10 df, p=<2e-16

Score (logrank) test = 112.8 on 10 df, p=<2e-16

```

time <- data_scen3$time
event <- data_scen3$event
X <- data_scen3 %>% dplyr::select(starts_with("X"))
```

### 3.3 Variable importance

We define time horizons, maximum follow-up time and times at which to approximate integrals.

```

landmark_times <- quantile(time, c(0.25, 0.75))
approx_times <- sort(unique(c(0, time[event == 1], landmark_times)))
approx_times <- approx_times[approx_times <= max(landmark_times)]
length(approx_times)
```

```
[1] 317
```

We generated cross-fitting and sample-splitting folds.

```

folds <- survML::generate_folds(n, V, sample_split)
cf_folds <- folds$cf_folds
table(cf_folds)

```

```

cf_folds
  1  2  3  4  5  6  7  8  9 10
50 50 50 50 50 50 50 50 50 50

```

```

ss_folds <- folds$ss_folds
table(ss_folds)

```

```

ss_folds
  0  1
250 250

```

```

V0_stackG_preds_scen3 <- CV_generate_full_predictions_landmark(
  time = time,
  event = event,
  X = X,
  landmark_times = landmark_times,
  approx_times = approx_times,
  nuisance = "stackG",
  cf_folds = cf_folds)
# str(V0_stackG_preds_scen3)

CV_full_preds <- V0_stackG_preds_scen3$CV_full_preds
CV_full_preds_train <- V0_stackG_preds_scen3$CV_full_preds_train
CV_S_preds <- V0_stackG_preds_scen3$CV_S_preds
CV_G_preds <- V0_stackG_preds_scen3$CV_G_preds

indx <- 1:ncol(X)
vim_stackG_scen3 <- purrr::map_dfr(indx, function(indx_i) {

  char_indx <- as.character(indx_i)
  indx <- as.numeric(strsplit(char_indx, split = ",")[[1]])

  CV_reduced_preds <- CV_generate_reduced_predictions_landmark(
    time = time,
    event = event,
    X = X,

```



```

    landmark_times = landmark_times,
    cf_folds = cf_folds,
    indx = indx,
    full_preds_train = CV_full_preds_train
  )

  purrr::map_dfr(vims, function(vim) {

    output <- switch(
      vim,
      brier = survML::vim_brier(
        time = time,
        event = event,
        approx_times = approx_times,
        landmark_times = landmark_times,
        f_hat = CV_full_preds,
        fs_hat = CV_reduced_preds,
        S_hat = CV_S_preds,
        G_hat = CV_G_preds,
        cf_folds = cf_folds,
        sample_split = sample_split,
        ss_folds = ss_folds
      ),
      AUC = survML::vim_AUC(
        time = time,
        event = event,
        approx_times = approx_times,
        landmark_times = landmark_times,
        f_hat = purrr::map(CV_full_preds, ~ 1 - .x),
        fs_hat = purrr::map(CV_reduced_preds, ~ 1 - .x),
        S_hat = CV_S_preds,
        G_hat = CV_G_preds,
        cf_folds = cf_folds,
        sample_split = sample_split,
        ss_folds = ss_folds
      )
    )

    output %>%
      dplyr::mutate(
        vim = vim,
        indx = char_indx
      )
  })

```

```

    )
  })
})

kable(vim_stackG_scen3 %>%
  dplyr::filter(vim == "AUC")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%
  dplyr::relocate(indx, landmark_time),
  col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large AUC", "small

```

| Variable | Horizon | VIM    | var(VIM) | CIL    | CIU   | p     | large<br>AUC | small<br>AUC |
|----------|---------|--------|----------|--------|-------|-------|--------------|--------------|
| 1        | 1.647   | -0.023 | 48.023   | -0.882 | 0.836 | 0.521 | 0.616        | 0.639        |
| 1        | 8.959   | -0.040 | 2.296    | -0.227 | 0.148 | 0.660 | 0.630        | 0.669        |
| 2        | 1.647   | -0.037 | 48.404   | -0.899 | 0.826 | 0.533 | 0.616        | 0.653        |
| 2        | 8.959   | -0.038 | 2.292    | -0.226 | 0.149 | 0.655 | 0.630        | 0.668        |
| 3        | 1.647   | -0.029 | 48.609   | -0.894 | 0.835 | 0.527 | 0.616        | 0.645        |
| 3        | 8.959   | -0.045 | 2.282    | -0.232 | 0.143 | 0.680 | 0.630        | 0.675        |
| 4        | 1.647   | -0.035 | 48.474   | -0.898 | 0.828 | 0.532 | 0.616        | 0.651        |
| 4        | 8.959   | -0.034 | 2.289    | -0.222 | 0.153 | 0.641 | 0.630        | 0.664        |
| 5        | 1.647   | -0.036 | 48.934   | -0.903 | 0.832 | 0.532 | 0.616        | 0.651        |
| 5        | 8.959   | -0.032 | 2.278    | -0.219 | 0.155 | 0.631 | 0.630        | 0.662        |
| 6        | 1.647   | 0.063  | 53.601   | -0.845 | 0.970 | 0.446 | 0.616        | 0.553        |
| 6        | 8.959   | 0.068  | 2.841    | -0.141 | 0.276 | 0.263 | 0.630        | 0.562        |
| 7        | 1.647   | -0.023 | 48.219   | -0.883 | 0.838 | 0.520 | 0.616        | 0.638        |
| 7        | 8.959   | -0.052 | 2.398    | -0.244 | 0.140 | 0.703 | 0.630        | 0.682        |
| 8        | 1.647   | -0.025 | 48.275   | -0.887 | 0.836 | 0.523 | 0.616        | 0.641        |
| 8        | 8.959   | -0.007 | 2.557    | -0.205 | 0.191 | 0.528 | 0.630        | 0.637        |
| 9        | 1.647   | -0.028 | 47.995   | -0.887 | 0.831 | 0.525 | 0.616        | 0.644        |
| 9        | 8.959   | -0.074 | 2.278    | -0.261 | 0.113 | 0.781 | 0.630        | 0.704        |
| 10       | 1.647   | -0.030 | 43.337   | -0.846 | 0.787 | 0.528 | 0.616        | 0.645        |
| 10       | 8.959   | -0.036 | 2.209    | -0.220 | 0.148 | 0.649 | 0.630        | 0.666        |

```

kable(vim_stackG_scen3 %>%
  dplyr::filter(vim == "brier")%>%
  tibble::remove_rownames()%>%
  mutate(across(c(landmark_time:small_predictiveness), ~round(.x, 3)))%>%
  dplyr::select(-c(cil_1sided, vim))%>%

```

```
dplyr::relocate(indx, landmark_time),
col.names = c("Variable", "Horizon", "VIM", "var(VIM)", "CIL", "CIU", "p", "large Brier", "small Brier")
```

| Variable | Horizon | VIM    | var(VIM) | CIL    | CIU   | p     | large Brier | small Brier |
|----------|---------|--------|----------|--------|-------|-------|-------------|-------------|
| 1        | 1.647   | -0.015 | 0.370    | -0.090 | 0.061 | 0.647 | -0.766      | -0.751      |
| 1        | 8.959   | -0.008 | 0.253    | -0.070 | 0.054 | 0.598 | -0.459      | -0.451      |
| 2        | 1.647   | -0.014 | 0.370    | -0.090 | 0.061 | 0.645 | -0.766      | -0.751      |
| 2        | 8.959   | -0.008 | 0.253    | -0.070 | 0.054 | 0.598 | -0.459      | -0.451      |
| 3        | 1.647   | -0.015 | 0.370    | -0.091 | 0.060 | 0.655 | -0.766      | -0.750      |
| 3        | 8.959   | -0.008 | 0.253    | -0.070 | 0.055 | 0.596 | -0.459      | -0.451      |
| 4        | 1.647   | -0.015 | 0.370    | -0.090 | 0.061 | 0.647 | -0.766      | -0.751      |
| 4        | 8.959   | -0.008 | 0.253    | -0.070 | 0.055 | 0.595 | -0.459      | -0.451      |
| 5        | 1.647   | -0.014 | 0.370    | -0.090 | 0.061 | 0.644 | -0.766      | -0.751      |
| 5        | 8.959   | -0.009 | 0.252    | -0.071 | 0.054 | 0.608 | -0.459      | -0.450      |
| 6        | 1.647   | -0.017 | 0.371    | -0.093 | 0.058 | 0.674 | -0.766      | -0.748      |
| 6        | 8.959   | -0.020 | 0.248    | -0.081 | 0.042 | 0.735 | -0.459      | -0.439      |
| 7        | 1.647   | -0.015 | 0.370    | -0.090 | 0.061 | 0.648 | -0.766      | -0.751      |
| 7        | 8.959   | -0.008 | 0.252    | -0.070 | 0.055 | 0.594 | -0.459      | -0.451      |
| 8        | 1.647   | -0.014 | 0.370    | -0.090 | 0.061 | 0.646 | -0.766      | -0.751      |
| 8        | 8.959   | -0.008 | 0.253    | -0.070 | 0.054 | 0.598 | -0.459      | -0.451      |
| 9        | 1.647   | -0.015 | 0.370    | -0.090 | 0.061 | 0.650 | -0.766      | -0.751      |
| 9        | 8.959   | -0.007 | 0.253    | -0.069 | 0.055 | 0.586 | -0.459      | -0.452      |
| 10       | 1.647   | -0.014 | 0.370    | -0.090 | 0.061 | 0.647 | -0.766      | -0.751      |
| 10       | 8.959   | -0.007 | 0.253    | -0.070 | 0.055 | 0.593 | -0.459      | -0.451      |

## 4 Additional numerical experiments

### 4.1 Sensitivity to covariate correlations

### 4.2 Exclusion versus permutation variable importance